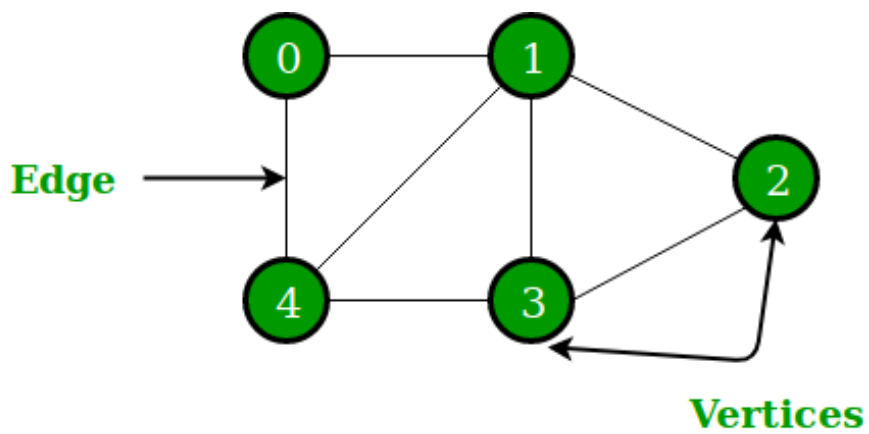
Graph

- Terminology
  - Nodes in trees are *vertices* in graphs.
  - Lines connecting vertices are *edges*.
  - Each edge is bounded by two vertices at its ends.
  - Two vertices are said to be *adjacent* to one another if they are connected by a single edge.



set of vertices V = {0,1,2,3,4} and the set of edges E = {01, 12, 23, 34, 04, 14, 13}.

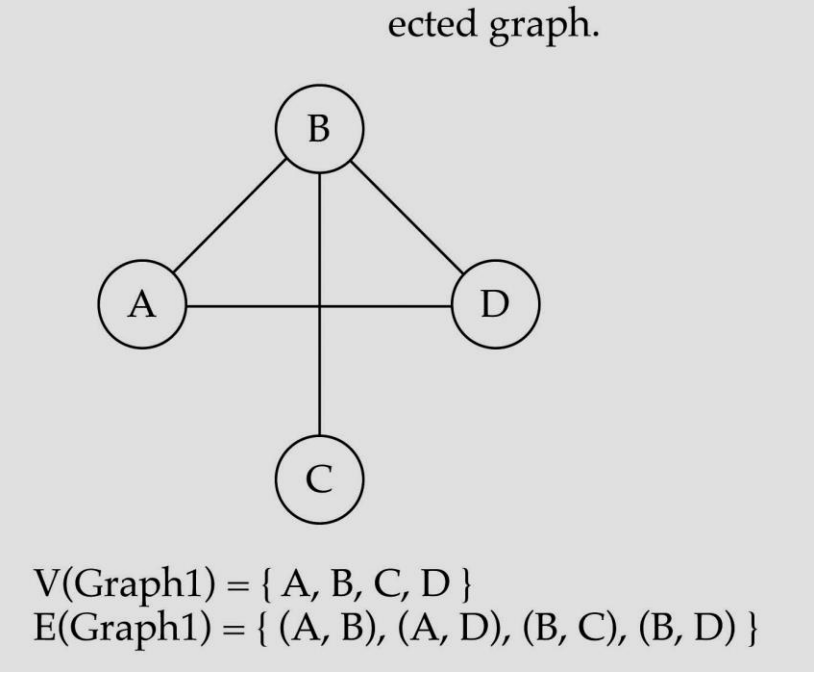
Definition

- A graph G consists of two sets
  - a finite, nonempty set of vertices V(G)
  - a finite, possible empty set of edges E(G)
  - G(V,E) represents a graph
- An undirected graph is one in which the pair of vertices in a edge is unordered, $(v_0, v_1) = (v_1, v_0)$
- A directed graph is one in which each edge is a directed pair of vertices, $<v_0, v_1> \; != \; <v_1, v_0>$

tail $\longrightarrow$ head

Directed vs. undirected graphs

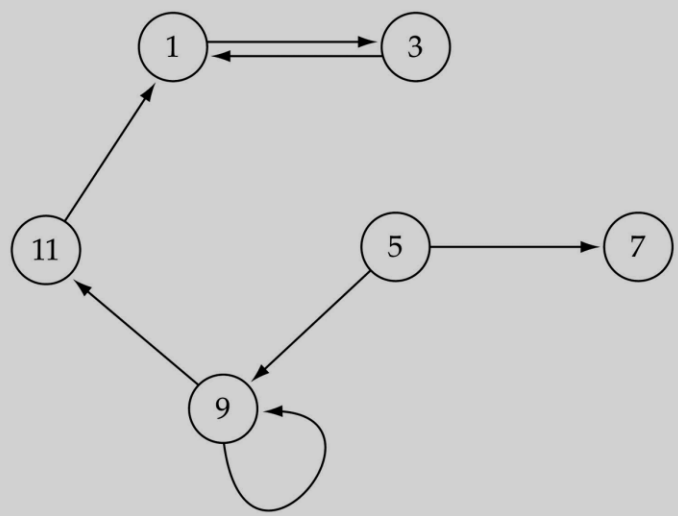
- When the edges in a graph have no direction, the graph is called *undirected*



Directed vs. undirected graphs (cont.)

- When the edges in a graph have a direction, the graph is called *directed* (or *digraph*)

(b) Graph2 is a directed graph.

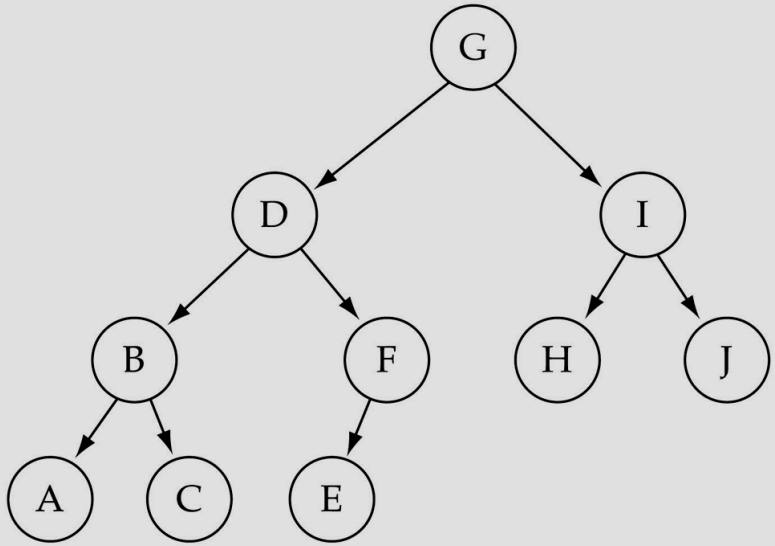


V(Graph2) = { 1, 3, 5, 7, 9, 11 }
E(Graph2) = {(1,3) (3,1) (5,9) (9,11) (5,7) 1), (9, 9), (11, 1) }

Trees vs graphs

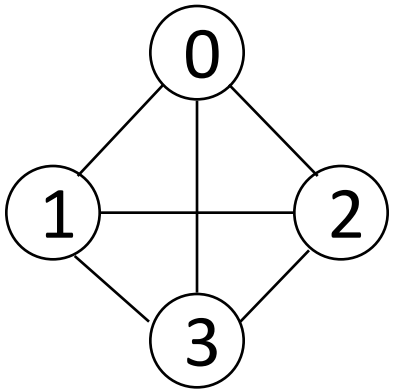
- Trees are special cases of graphs!!
- In trees- no cycle

(c) Graph3 is a directed graph.



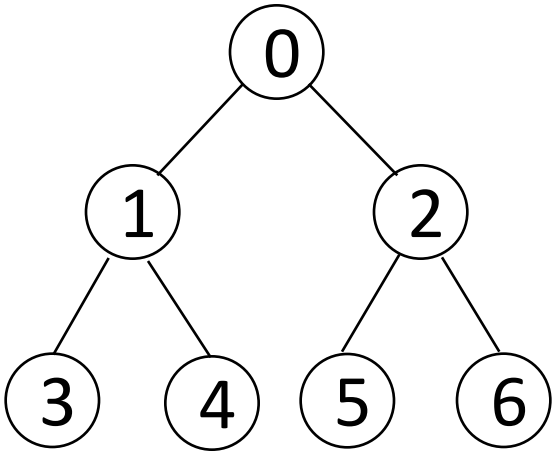
V(Graph3) = { A, B, C, D, E, F, G, H, I, J }
E(Graph3) = { (G, D), (G, J), (D, B), (D, F) (I, H), (I, J), (B, A), (B, C), (F, E) }

Examples for Graph



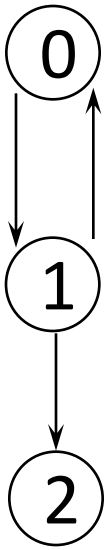
G_1

complete graph



G_2

incomplete graph



G_3

$V(G_1)=\{0,1,2,3\}$ $E(G_1)=\{(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)\}$

$V(G_2)=\{0,1,2,3,4,5,6\}$ $E(G_2)=\{(0,1),(0,2),(1,3),(1,4),(2,5),(2,6)\}$

$V(G_3)=\{0,1,2\}$ $E(G_3)=\{<0,1>,<1,0>,<1,2>\}$

complete undirected graph: n(n-1)/2 edges

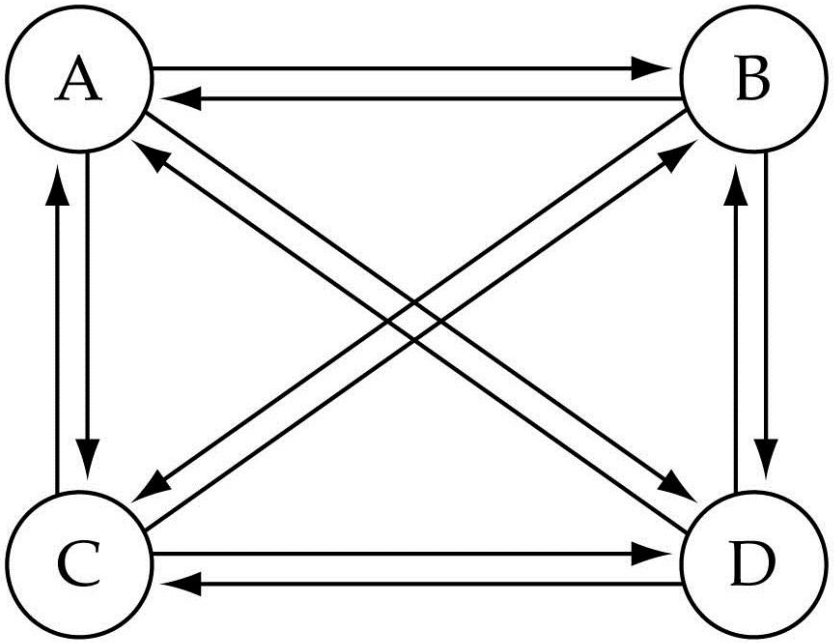
complete directed graph: n(n-1) edges

Graph terminology (cont.)

- What is the number of edges in a complete directed graph with N vertices?

*N * (N-1)*

$O(N^2)$



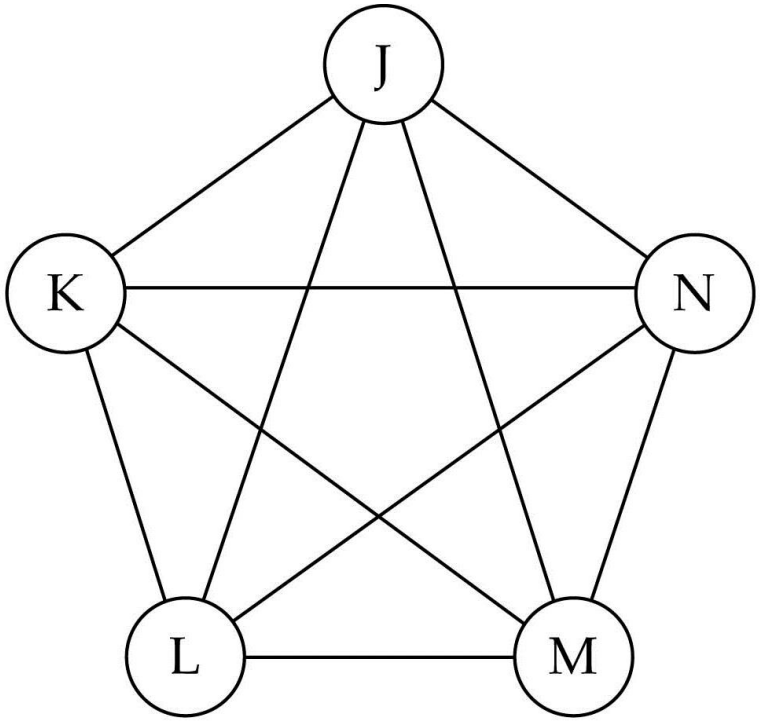
(a) Complete directed graph.

Graph terminology (cont.)

- What is the number of edges in a complete undirected graph with N vertices?

*N * (N-1) / 2*

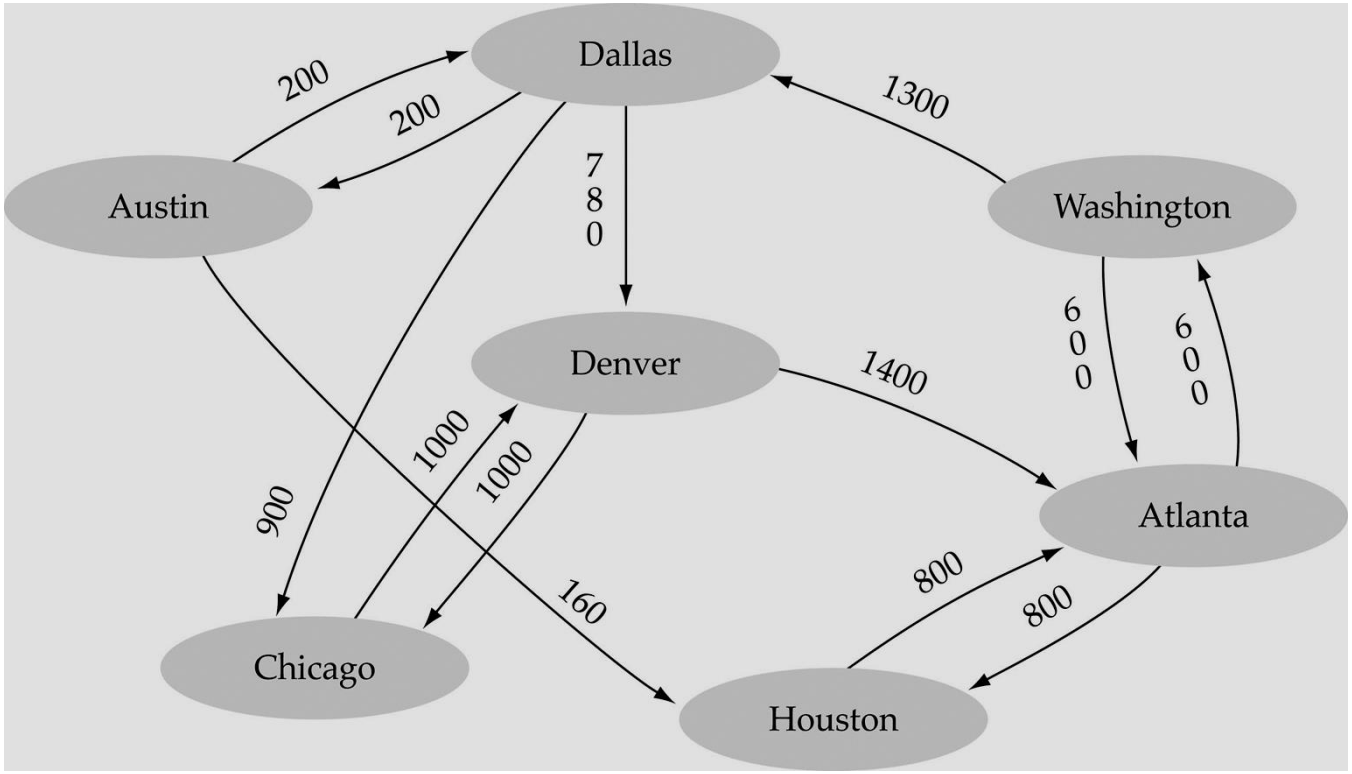
$O(N^2)$



(b) Complete undirected graph.

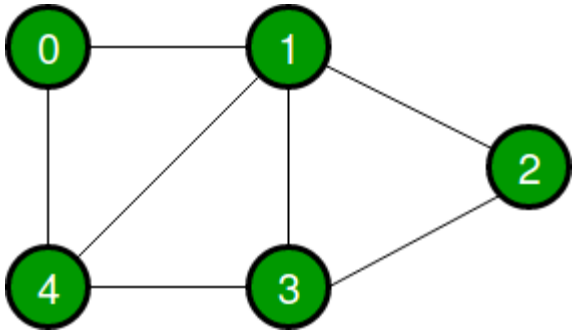
Graph terminology (cont.)

- <u>Weighted graph</u>: a graph in which each edge carries a value



Adjacency matrix vs. adjacency list representation

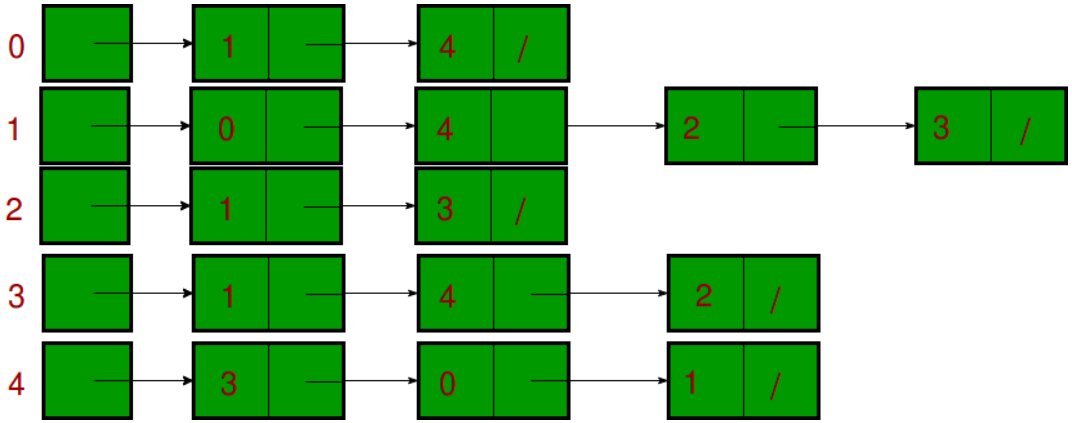
- **Adjacency matrix**
  - Good for dense graphs --$|E| \sim O(|V|^2)$
  - Memory requirements: $O(|V| + |E|) = O(|V|^2)$
  - Connectivity between two vertices can be tested quickly
- **Adjacency list**
  - Good for sparse graphs -- $|E| \sim O(|V|)$
  - Memory requirements: $O(|V| + |E|) = O(|V|)$
  - Vertices adjacent to another vertex can be found quickly



- **Adjacency Matrix**

	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

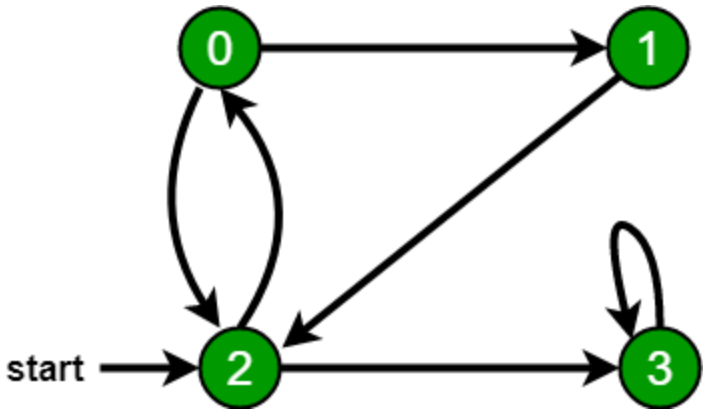
- **Adjacency List**



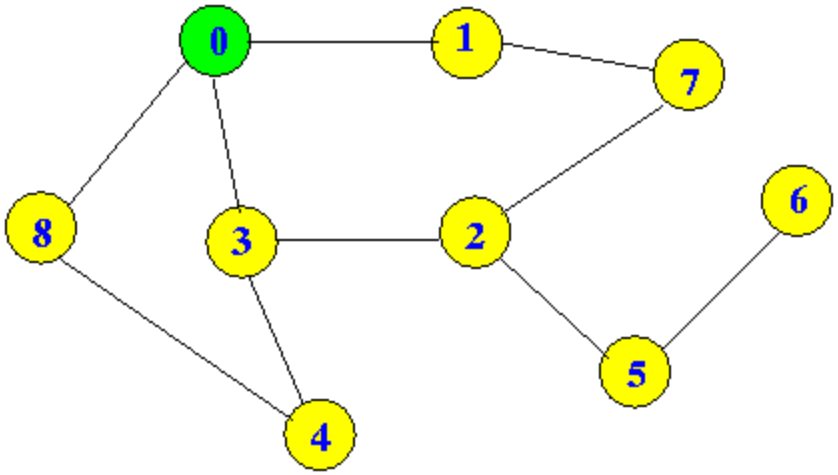
Traversal

- BFS(Breadth First Search
- DFS(Depth First Search)

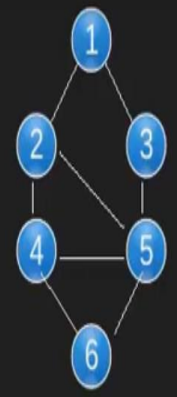
BFS



2 0 3 1

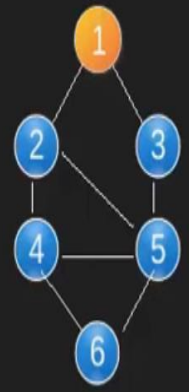


0 1 3 8 7 2 4 5 6



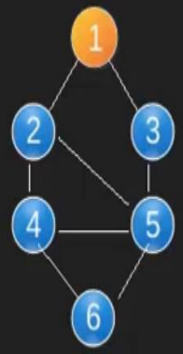
	1	2	3	4	5	6
Visited :	0	0	0	0	0	0

Queue :



	1	2	3	4	5	6
Visited :	1	0	0	0	0	0

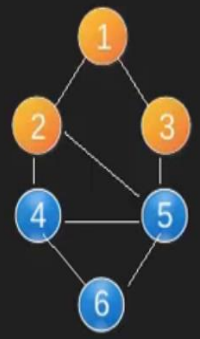
Queue : 1



	1	2	3	4	5	6
Visited :	1	0	0	0	0	0

Queue :

Print : 1

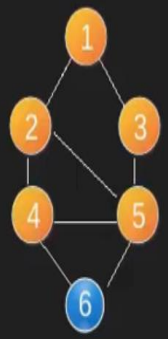


	1	2	3	4	5	6
Visited :	1	1	1	0	0	0

Queue : 2 3

After removing 1 from queue and printing it, we enqueue its non-visited adjacent Nodes

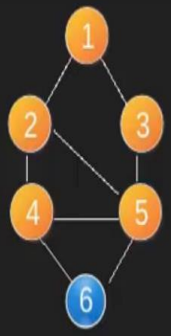
Print : 1



	1	2	3	4	5	6
Visited :	1	1	1	1	1	0

Queue : 3 4 5

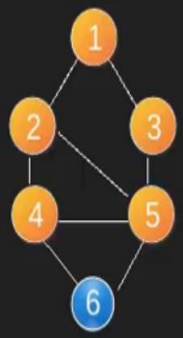
Print : 1 2



	1	2	3	4	5	6
Visited :	1	1	1	1	1	0

Queue : 4 5

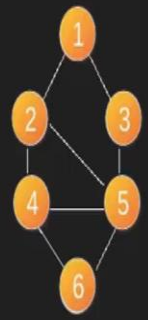
Print : 1 2 3



	1	2	3	4	5	6
Visited :	1	1	1	1	1	0

Queue : 5

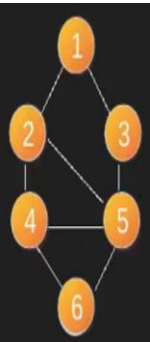
Print : 1 2 3 4



	1	2	3	4	5	6
Visited :	1	1	1	1	1	1

Queue : 5 6

Print : 1 2 3 4



	1	2	3	4	5	6
Visited :	1	1	1	1	1	1

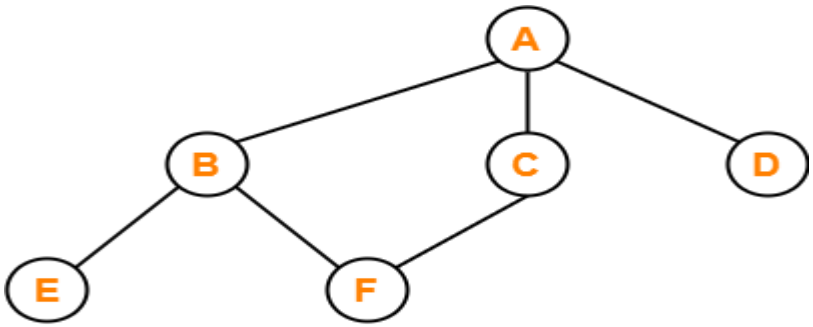
Queue : 6

Print : 1 2 3 4 5

- Time Complexity: O(V+E)
- Applications of Breadth First Traversal

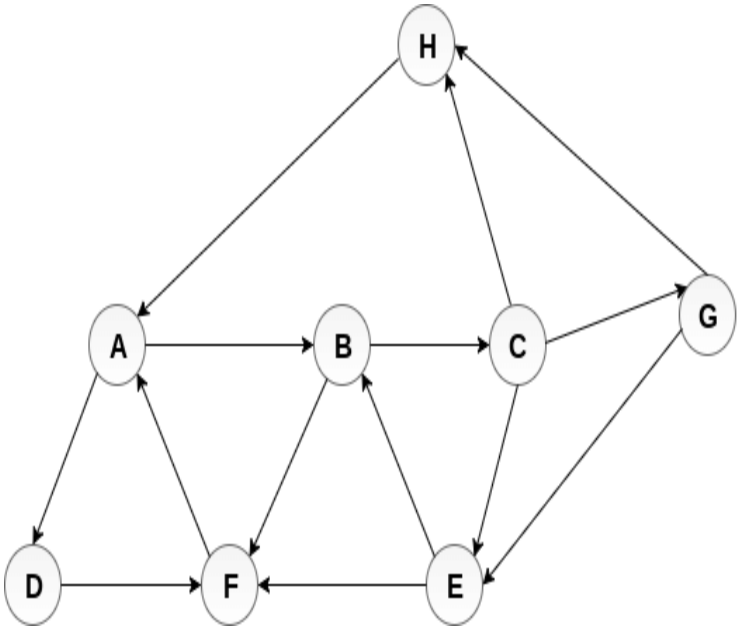
Shortest Path and Minimum Spanning Tree for unweighted graph, Peer to Peer Networks, Crawlers in Search Engines, Social Networking Websites, GPS Navigation systems, Garbage Collection

DFS



- ABEFCD

Depth First Search Example



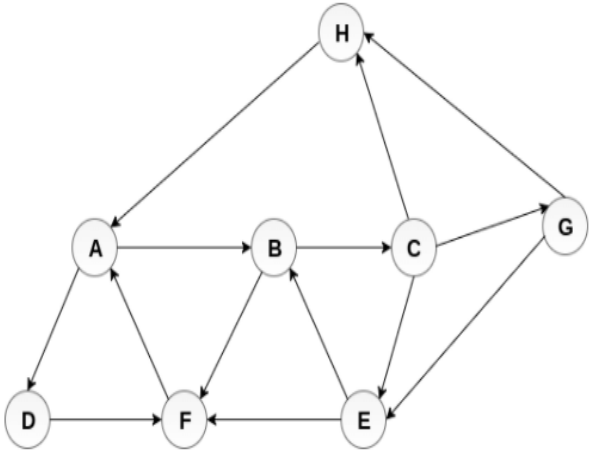
Adjacency Lists

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

- HADFBCGE

Algorithm

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Push the starting node A on the stack and set its STATUS = 2 (waiting state)
- **Step 3:** Repeat Steps 4 and 5 until STACK is empty
- **Step 4:** Pop the top node N. Process it and set its STATUS = 3 (processed state)
- **Step 5:** Push on the stack all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)
  [END OF LOOP]
- **Step 6:** EXIT



Adjacency Lists

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

Solution :

Push H onto the stack

```
STACK : H
```

POP the top element of the stack i.e. H, print it and push all the neighbours of H onto the stack that are is ready state.

```
Print H
STACK : A
```

Pop the top element of the stack i.e. A, print it and push all the neighbours of A onto the stack that are in ready state.

```
Print A
Stack : B, D
```

Pop the top element of the stack i.e. D, print it and push all the neighbours of D onto the stack that are in ready state.

```
Print D
Stack : B, F
```

Pop the top element of the stack i.e. F, print it and push all the neighbours of F onto the stack that are in ready sta

```
Print F
Stack : B
```

Pop the top of the stack i.e. B and push all the neighbours

```
Print B
Stack : C
```

Pop the top of the stack i.e. C and push all the neighbours.

```
Print C
Stack : E, G
```

Pop the top of the stack i.e. G and push all its neighbours.

```
Print G
Stack : E
```

Pop the top of the stack i.e. E and push all its neighbours.

```
Print E
Stack :
```

Hence, the stack now becomes empty and all the nodes of the graph have been traversed.

The printing sequence of the graph will be :

```
H → A → D → F → B → C → G → E
```